Effect of Collective Communication Algorithms on LLM Training

Group-12

Anand	Arjun Prasad	Videh Sharma	Yash Verma
MT25014	MT25064	MT25052	MT25092

1. Abstract

Large Language Models are trained on large datasets. These LLMs can contain billions of parameters. As these models continue to grow in size, training on a single GPU becomes infeasible due to compute limitations. Modern LLM training relies on distributed training where the model and/or dataset is distributed across multiple GPUs. In distributed training, GPUs exchange information during the training process which is managed by ***collective communication operations*** such as *AllReduce*. However, evaluating the cost of LLM training in real-world environments is expensive as it requires hardware infrastructure and time consuming to set up. To address this, **LLM simulators** are developed. Simulators replicate *behaviour* of distributed training systems without actual hardware deployments.These simulators run across different configurations to study the effect of different parameters on training the LLMs.

2. Introduction

Modern LLMs contain billions or even trillions of parameters. The training of LLMs is performed using multiple GPUs operating in parallel, where the training workload is distributed across devices. Each GPU receives a portion of the dataset and performs training on it. Since model parameters must remain consistent across GPUs, the gradients are shared and synchronized across all GPUs. This is known as *collective communication.*

Collective communication refers to the communication operations which exchange data (eg : gradients/output activations) across GPUs. They play a central role in distributed learning as they allow consistency across training.

Some commonly used collective communication operations are *AllReduce* and *AllGather*. In AllReduce operation, each GPU computes gradients on its given batch of data and then a reduce operation aggregates all the gradients across all GPUs typically by either summing or averaging. The final aggregate gradient is distributed across all GPUs. This is commonly used in *data parallelism* training strategy.

In AllGather the each of the GPU is trained on to smaller portions of the data and afterwards each of the GPU shares the tensors(containing Gradients,Model weights) with one another and uses these tensors for the further computation. This is commonly deployed in *tensor parallelism*.

There are several algorithms used to implement the collective communication operation. They describe how the data is being transferred between different GPUs. *Ring-AllReduce* and *Tree-AllReduce* are some popularly known algorithms. In the Ring-AllReduce the GPU's are arranged in the form of a ring where the data is circulated in a ring between different GPUs. In *Tree-AllReduce* GPUs are arranged in the tree structure and the gradients transfer from the leaf node to the root node.

3. Problem

As the size of the system increases with more GPUs it comes with challenges such as making communication efficient,reducing the training time etc. The simulator evaluates performance of the different algorithms such as Ring-AllReduce and Tree-AllReduce with varying parameters such as message size, number of GPUs etc.The simulator finds the impact on training time, efficiency and communication latency. Our Aim is to compare the performance of the system with varying parameters. And to find which algorithm works optimally under the different scenarios. The study helps to identify how different algorithms influence LLM training performance under varying conditions.

4. Motivation

The scale of the LLM training problem is substantial. For example, training the model in GPT-3 required approximately 34 days on 1024 GPUs. Even small improvements in communication efficiency has a significant impact on the training.Even small improvement in communication performance can reduce training time by several days.
Conducting controlled experiments on real clusters at this scale is expensive and infeasible. This motivates the use of simulation-based approaches to understand LLM training. Simulators such as SimAI replicate the behavior of large-scale AI workloads and distributed training systems under different configurations. Its reported accuracy levels of around 98.1% compared to real systems enable researchers to study the performance impact of different collective communication algorithms without requiring access to real-hardware clusters. As a result, simulation-driven experimentation provides a cost-effective way to generate insights that can inform real-world system design and optimization.

5. Objectives

We aim to study the following objectives through this simulation:

1. Performance of Collective Algorithms:
   To compare the performance of Ring-AllReduce and Tree-AllReduce collective communication algorithms as a function of parameters such as message sizes, number of GPUs, network bandwidth.
2. Optimal Operating Strategies:
   To identify optimal operating conditions - such as defined by combination of different parameters such as message size, GPU count etc - in which each collective communication algorithm provides performance.
3. Effect of Parallelism Strategies:

To analyze how distributed training parallelism strategies - tensor parallelism , data parallelism and pipeline parallelism influence choice of collective communication algorithms.

6. Background and related work

6.1 LLM training

Large Language Models (LLMs) like GPT-4 have around 1.8 trillion parameters and require approximately 25,000 GPUs for training. This enormous computational demand translates into significant financial costs and creates barriers to parallel development. For instance, decisions about which topology, network, or data collection algorithm to use—and under what constraints they perform best—are difficult to explore in practice. These challenges can limit the democratization of LLM training. This is where LLM simulators come in: they allow researchers and developers to simulate LLM training with far fewer computational resources, making experimentation more accessible and affordable.

LLM simulators address several key challenges:

1. Comprehensive evaluation of AI infrastructure from multiple perspectives
2. Cost-effective validation of optimizations
3. Development of a unified simulation framework
4. High-fidelity communication simulation
5. High-fidelity computation simulation
6. High simulation speed

6.2 Synchronization

One of the main steps in distributed training of LLMs is *gradient synchronization*. The primary objective of LLM training is to learn the model weights that minimize the loss function.

In an extensive training environment, the workload is distributed across multiple GPUs using data parallelism. Each GPU processes a different subset of the training data and computes gradients independently during

the backward pass. However, to ensure that all distributed part of the model remain consistent, the gradients computed on different GPUs must be synchronized before updating the model parameters.
To perform gradient synchronization efficiently, distributed training frameworks use collective communication operations, most commonly AllReduce. The AllReduce operation aggregates gradients from all GPUs and computes their average, after which each GPU updates its local copy of the model parameters.

6.3 Parallelisation strategies

Parallelisation strategies describe how the training workload of LLMs is distributed across multiple GPUs. Generally, three major parallelization strategies are used :

- **Data Parallelism (DP)** - Each GPU maintains a replica of the model and processes different batches of training data.
- **Tensor Parallelism (TP)** - Model tensors and computations within individual layers are partitioned across multiple GPUs.
- **Pipeline Parallelism (PP)** - Layers of DNN are distributed across all nodes.

Frameworks such as Megatron and DeepSpeed implement a combination of above strategies to support training.

6.4 Collective communication

After the training workload has been distributed across multiple GPUs using parallelization, GPUs exchange information (such as gradients or output activations) for consistent training. This is achieved through *collective communication operations*. One such operation is *AllReduce*, which aggregates gradients across all GPUs. Some of the algorithms which implement AllReduce is Ring-AllReduce and Tree-based AllReduce.

6.5 Overview of NCCL:

The NVIDIA Collective Communication Library (NCCL) is a library which provides utilities for inter-GPU communication. It provides implementations of collective communication algorithms such AllReduce and AllGather.

7. Methodology/Design

Several simulators have been proposed for analyzing large-scale training systems. Among these, SimAI offers a unified simulation framework that aims to accurately and efficiently model the large-scale training process of Large Language Models (LLMs).

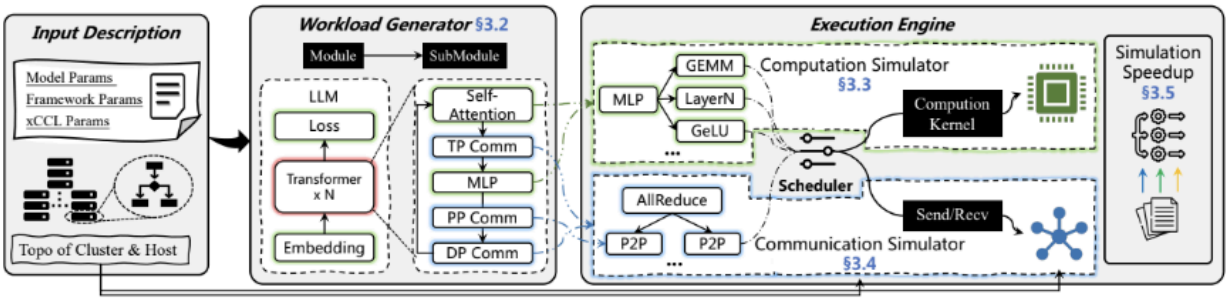


Figure 1: The SimAI architecture

7.1 Components of SimAI:

SimAI is a unified simulator specifically designed to model the training process of LLMs . Its architecture integrates several essential components to collectively simulate the key aspects of distributed training: computation, communication, and scheduling. These major components are detailed in the following section.

7.1.1 Input Description:

The input description provides as a starter for the simulation it contains detailed specification of the simulation environment and the parameters required to run the simulation. It basically includes configuration of the hardware environment, such as the number of GPUs, network bandwidth, and system characteristics.

Additionally, the input description specifies the network topology and other system-level parameters that influence communication and computation

during training. These parameters enable the simulator to replicate large-scale training environments in a controlled and configurable manner.

7.1.2 Workload Generator:

The SimAI Workload Generator (SimAI-WG) generates workloads that closely resemble real-world LLM training workloads. It achieves this by hijacking existing deep learning training frameworks to produce traces that mimic real training executions.

One of the primary challenges is enabling a single host system to behave as if it were operating within a large GPU cluster. To address this, certain modifications are made to the training framework.

Key approaches include:

i. Framework Modification

The framework is modified so that a single machine can emulate the behavior of a distributed environment containing the target number of GPUs. This allows the workload generator to produce traces representative of large-scale training systems

ii. NCCL Modification

The NCCL (NVIDIA Collective Communications Library) is modified to support pipeline parallelism within the simulated environment. The workload generator must also be configured with the appropriate rank numbers, which represent the logical identity of each GPU within the distributed system.

7.1.2.1 Module: This module broadly comprises three parts: Embedding, Transformer, and Loss calculation. Embeddings convert token IDs into vectors; Trans * N represents the number of Layers. 50 means it is expanded up to 50 Layers.

7.1.2.2 Loss: Loss has been used for training the model and backpropagation. The common Loss which we are using here is Cross-Entropy Loss, language modelling loss, etc.

7.1.2.3 Sub Modules: Now we are going to expand each transformer. Each transformer layer contains Self Attention, MLP, and Communication ops. All these parts combined to form a submodule, which repeats N times, giving the no. of layers involved.

7.1.2.4 Self Attention: Compute attention between the tokens so that it adds more meaning to the whole sentence now each word meaning and the word meaning in a sentence may differ for example “I live along a River Bank”, “I had deposited my money in a Bank” if look to the specific word embedding both bank looks same, but it is what the sentence which is going to create the whole difference hence in Self Attention, Attention is calculated between the tokens to add more meaning to the specific word.

7.1.2.5 Parallelisation: There are three kinds of parallelization involved: Tensor Parallelisation, Data Parallelisation, and Pipeline Parallelisation. All these parallelisation approaches are implemented to divide the data or the work between the layers

7.1.3 Execution Engine: It defines how the data is going to run on the hardware

7.1.3.1 Computation Simulator:
As the name suggests, this part is going to simulate the computations that happen during the training of an LLM. This will include GEMM (General Matrix Multiply), GeLU(an activation function), and LayerNorm(Normalises activations)

i-Computation kernel: Each operation is mapped to a hardware kernel. Simulating the simulator estimates Execution time, GPU utilizations, Flops Consumed, etc they are not real hardware kernels present here

7.1.3.2 Communication Simulator : There is continuous data movement between GPUs during training; they constantly exchange data, even during backpropagation. Now they all have to coordinate to implement the

algorithm properly, and we need to synchronize the gradient between all the GPUs.

- **All Reduce:-** used for gradient synchronization; it operates on the principle of averaging gradients.
- **P2P:-** peer-to-peer direct communication between GPU.
- **send/recv:-** is used to send and receive the tensor/gradients, and then work in accordance with those gradients .

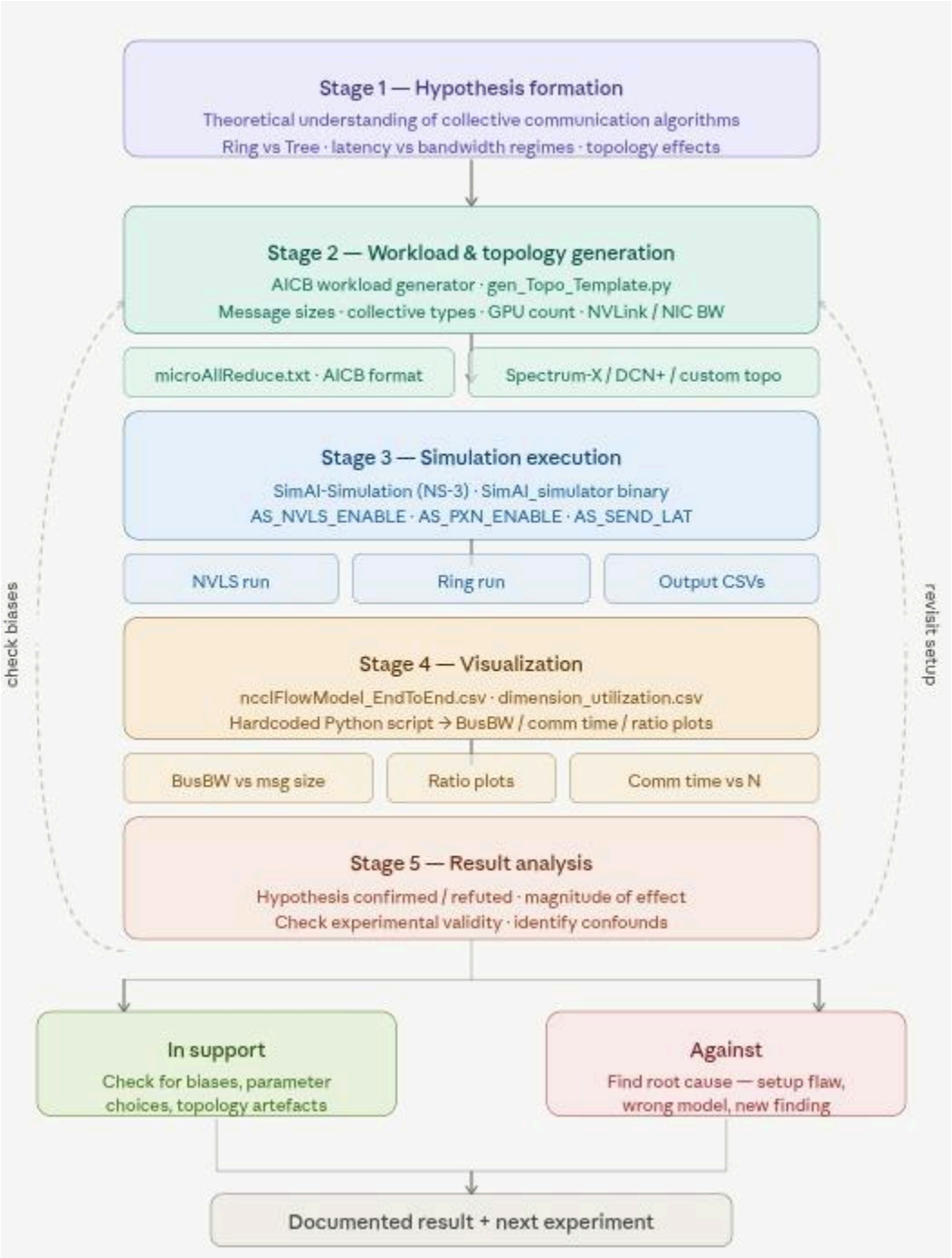
7.1.3.3 Scheduler : acts as an orchestrator; it determines the execution order, decides when communication runs, and defines the dependencies between tasks. It establishes the correct timeline for execution and coordination.

7.1.3.4 Simulation SpeedUp :-We can recall that models like GPT take about **34 days to train**. However, many times we are not interested in obtaining the final trained weights. Instead, we are interested in identifying the parameters on which the training process depends and understanding how training performance varies with these parameters. Therefore, we require faster experimentation, which can be achieved through **simulation speedup**.

The techniques used in simulation speedup include

- **Kernel time modelling instead of real execution** .
- **Parallel event simulation**.
- **Trace compression .**

8. Implementation



Stage 1 — Hypothesis Formation

The starting point is always a claim grounded from the theoretical point of view , We Documented the hypothesis formally before running anything and also the theoretical reason which supports the hypothesis has been discussed

Stage 2 — Workload and Topology Generation

Two things are compulsory before verifying any hypothesis these are the workload file , controls what operations run — collective type, message sizes, compute time, number of layers, parallelism group. The topology file controls the physical network — number of GPUs, NVLink bandwidth, NIC bandwidth, rail-optimization, switch configuration. These two together define the full experimental condition.

Stage 3 — Simulation Execution

The binary is `SimAI_simulator` with environment variables as the primary control surface. The three most important knobs are `AS_NVLS_ENABLE`, `AS_PXN_ENABLE`, and `AS_SEND_LAT`. Each run produces two CSVs: `ncclFlowModel_EndToEnd.csv` (per-layer timing and bandwidth) and `ncclFlowModel_detailed_*.csv` (per-node break down). Logs are also saved with `tee` for traceability.

Stage 4 — Visualization

The CSV data is extracted, all values hardcoded into a Python script, and then plotted. The graphs are plotted as per the need of the hypothesis and then verified

Stage 5 — Result Analysis

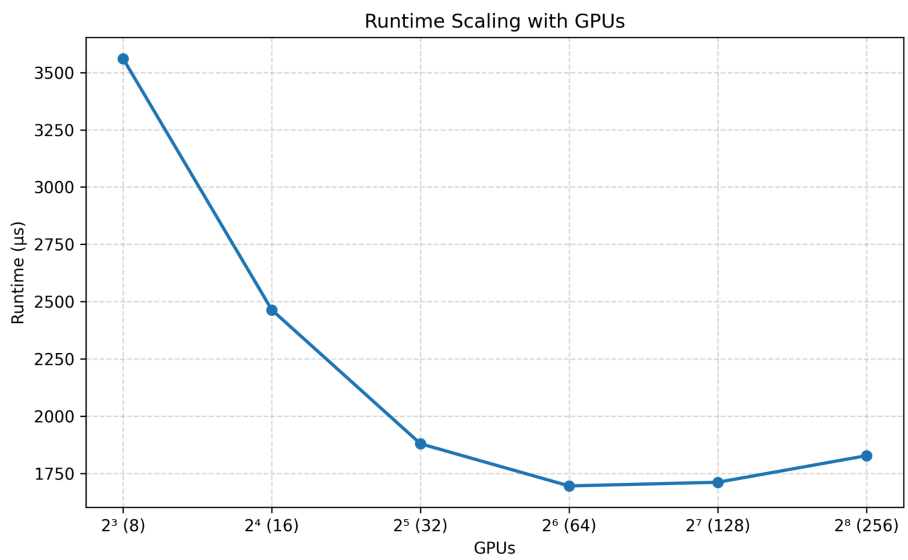
This is done in two parts basically whether its support the hypothesis or it does not support the hypothesis

If the result supports the hypothesis,then we will check for our workload and configurations whether its all correct and dont have bias towards the hypothesis after checking for it we go forward with the conclusion

If the result contradicts the hypothesis, there are two possible causes: either the experimental setup has a flaw , or the hypothesis is genuinely wrong. We first try to eliminate setup issues — compare your run commands against the tutorial reference, verify the topology file matches your intent, check that both algorithms are tested on equivalent conditions. If setup is clean, the contradiction is the finding and we move forward with it. If the hypothesis is correct and our results are wrong then we also try to write down the implementation constraints we are facing.

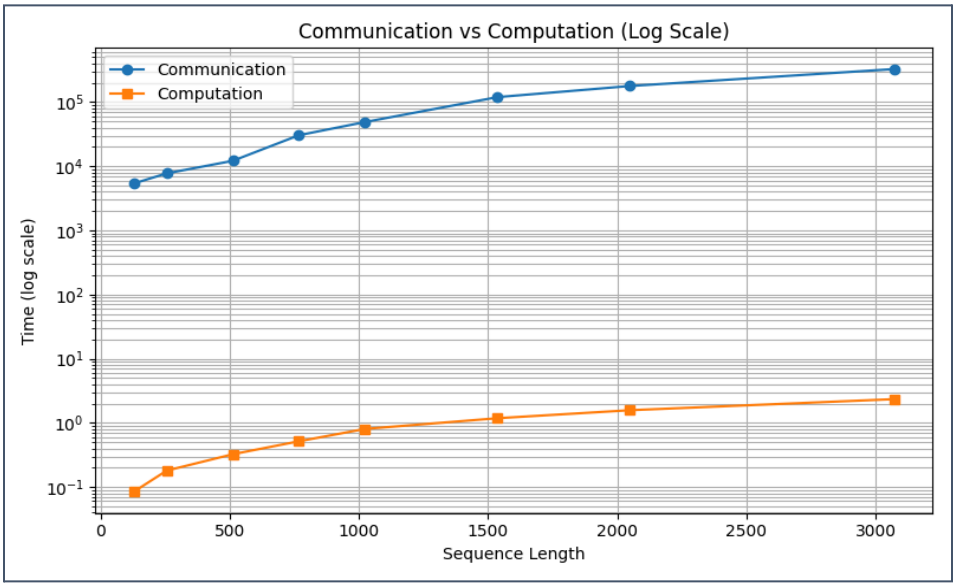
9. Observations

1.



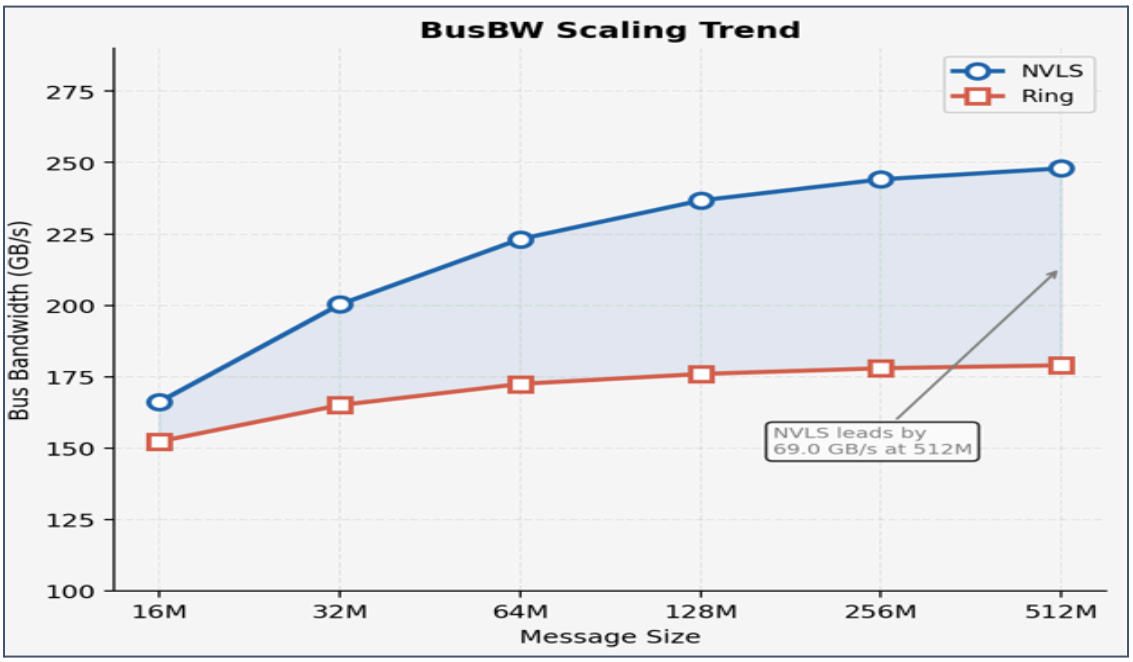
Hypothesis : Execution time should reduce with increase in GPUs

2.



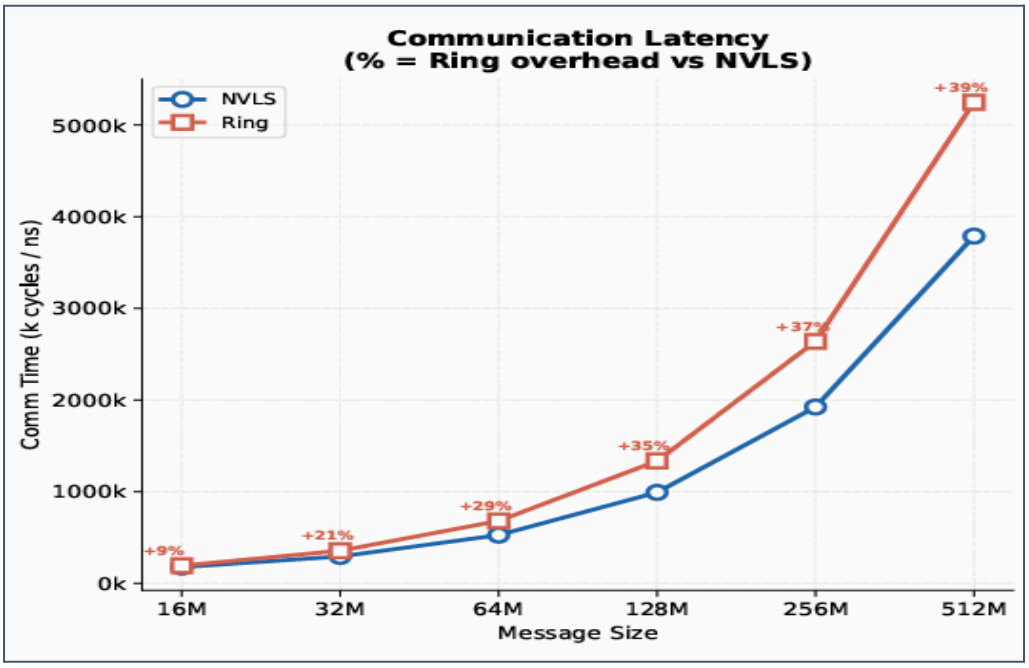
Hypothesis : At small sequence lengths, communication time dominates algorithm choice matters more.

3.



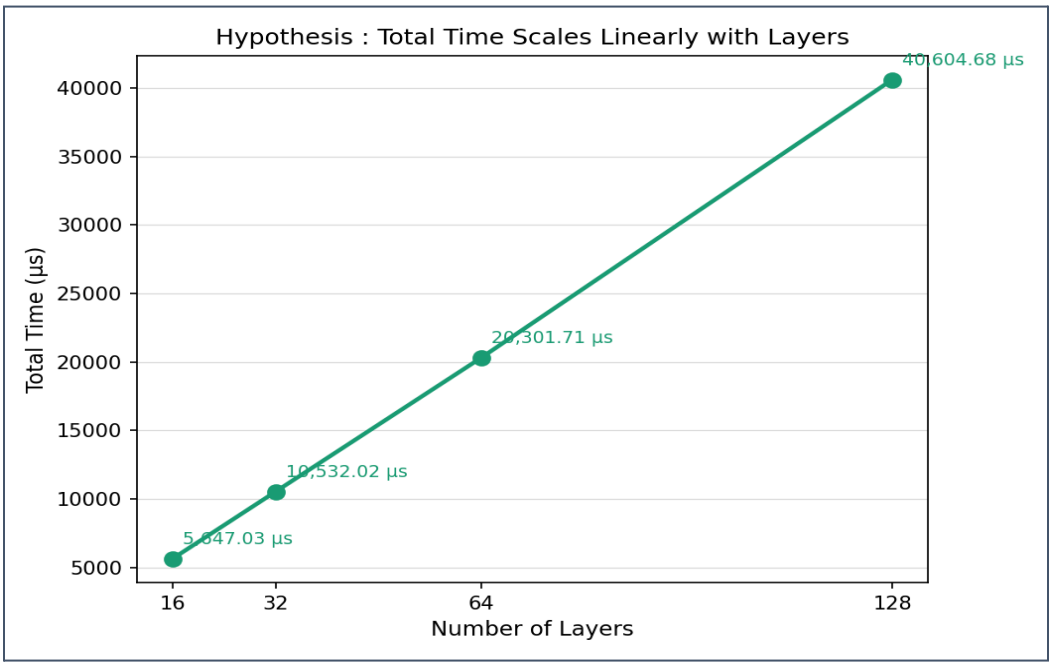
Hypothesis : As the Message size increases the Bus Bandwidth of the NVLS scales better as compared to the Ring.

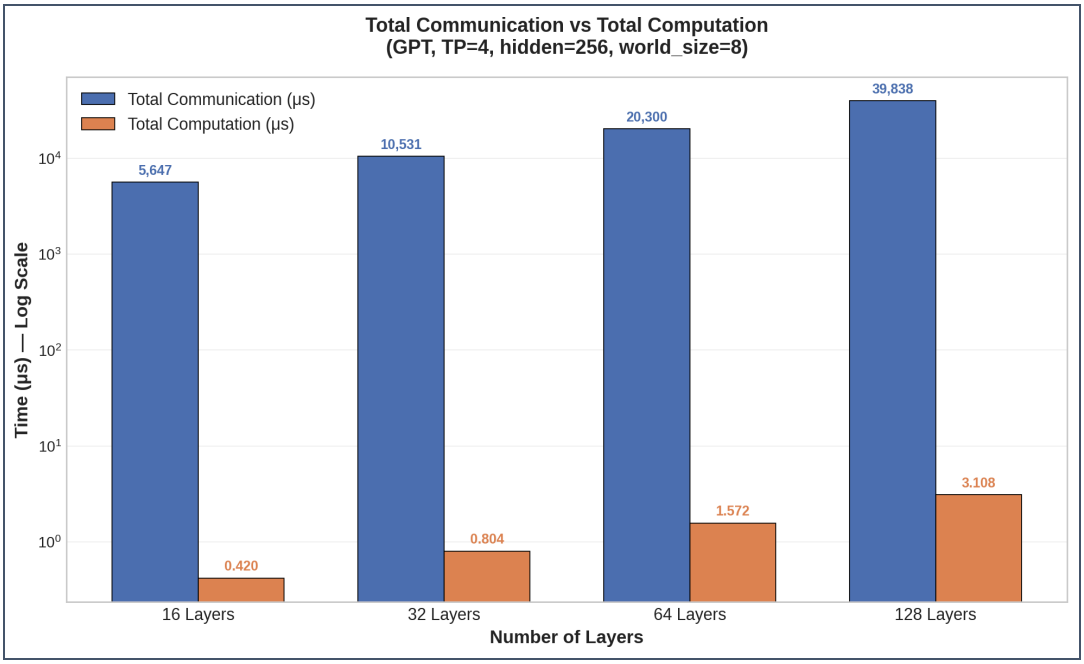
4.



Hypothesis : As the Message size increases the Communication Latency of NVLS is more compared to the Ring.

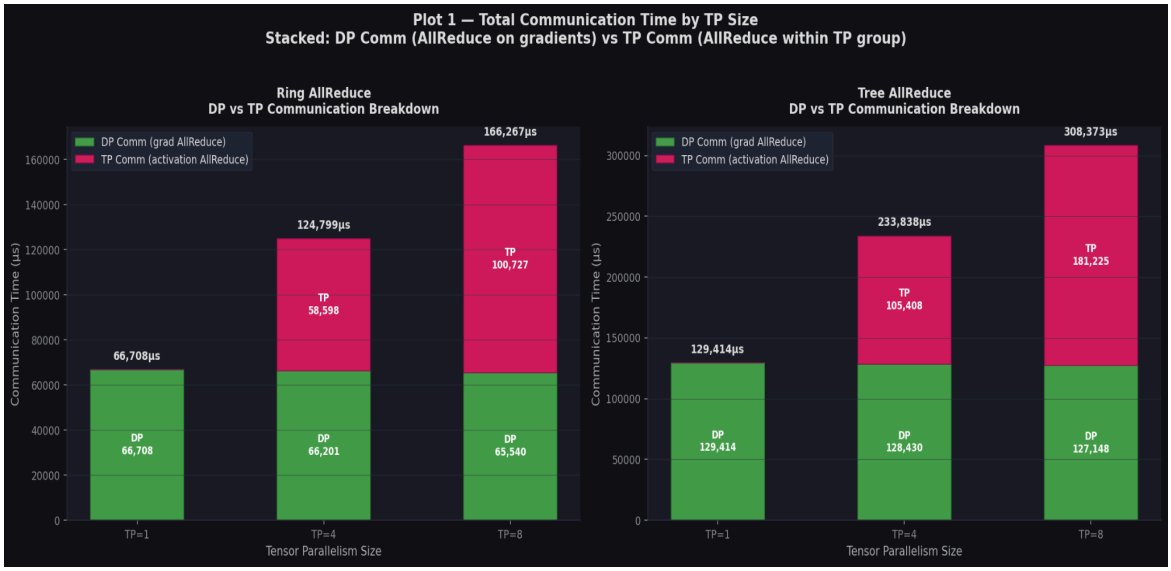
5.



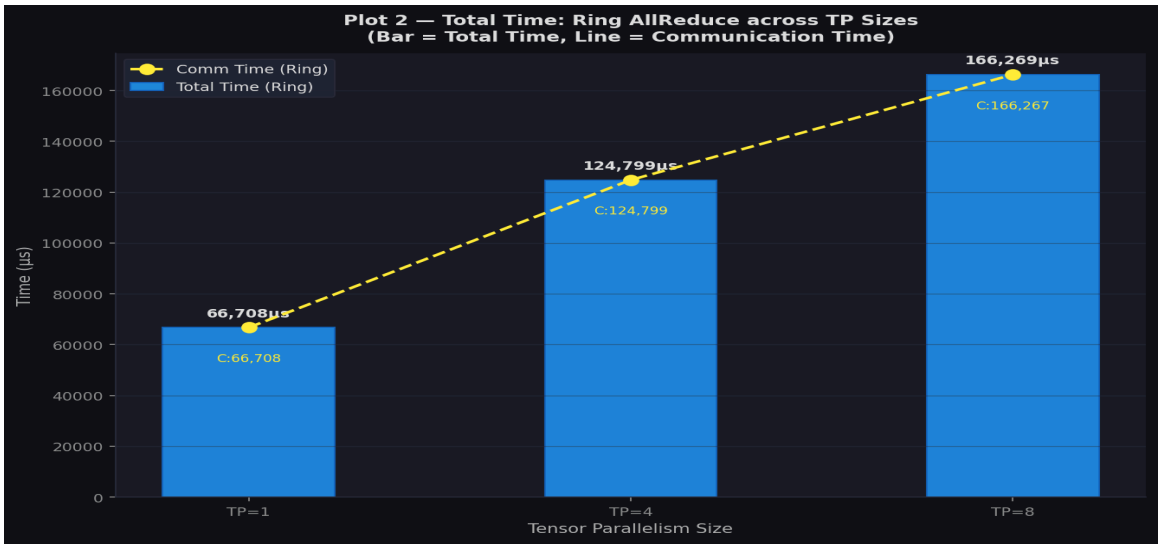


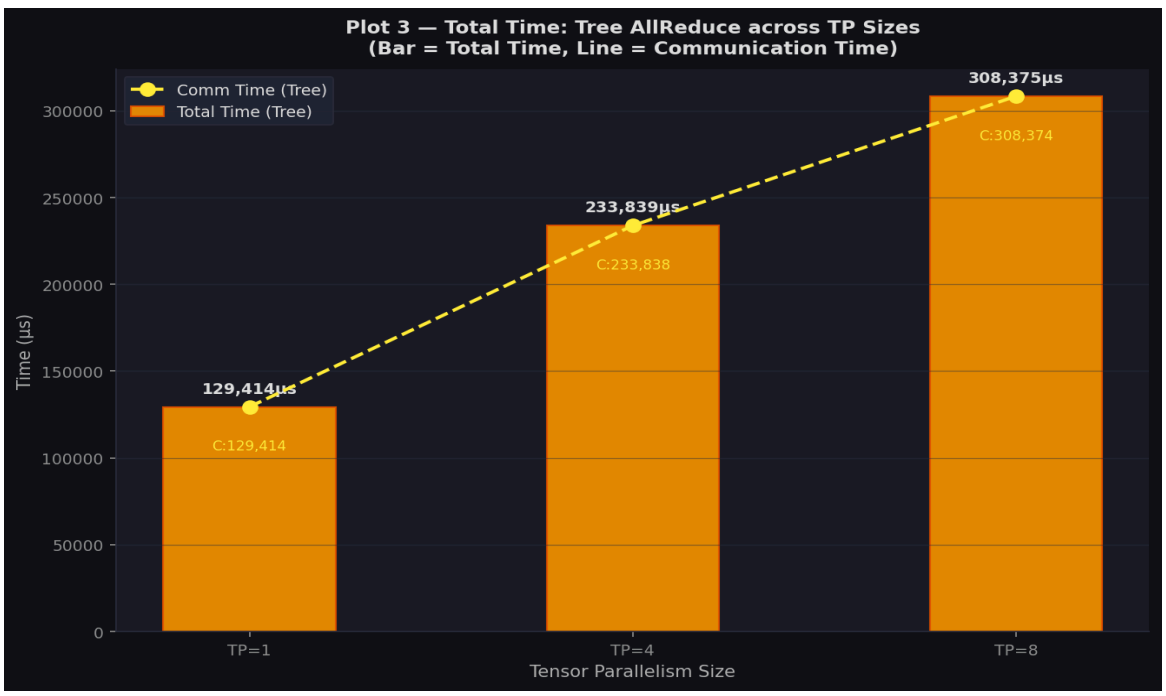
Hypothesis : `The total execution time scales linearly with the number of layers, provided that all other factors remain constant.`

6.



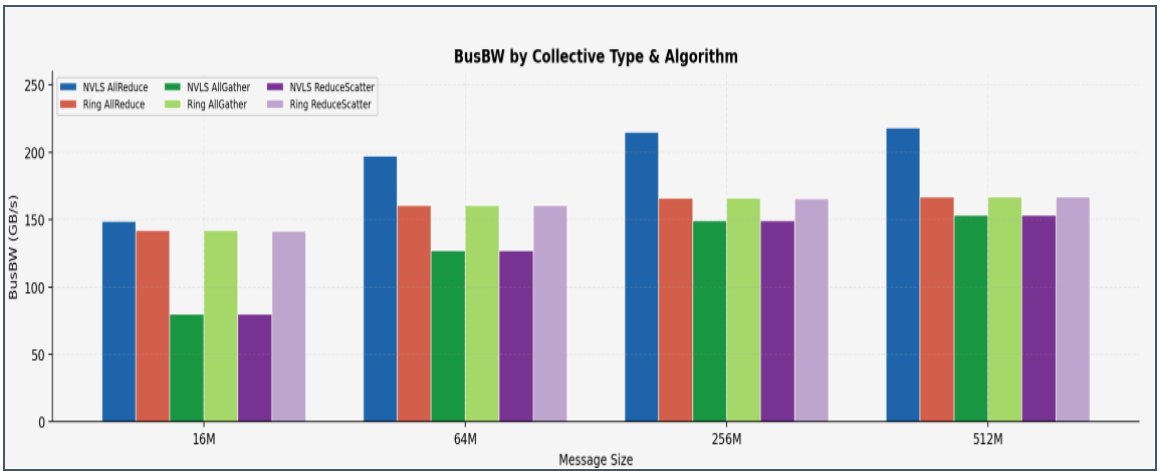
Hypothesis : Higher TP increases frequency and size of AllReduce → magnifies differences between algorithms.

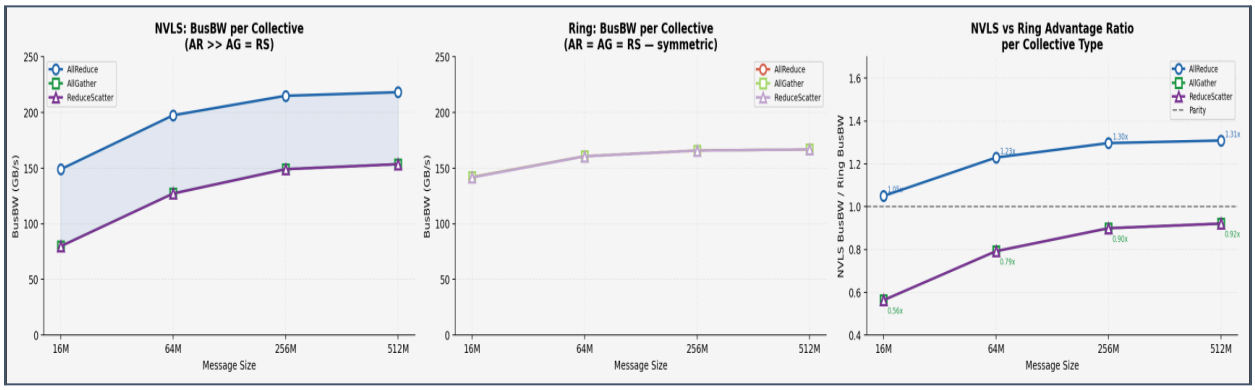


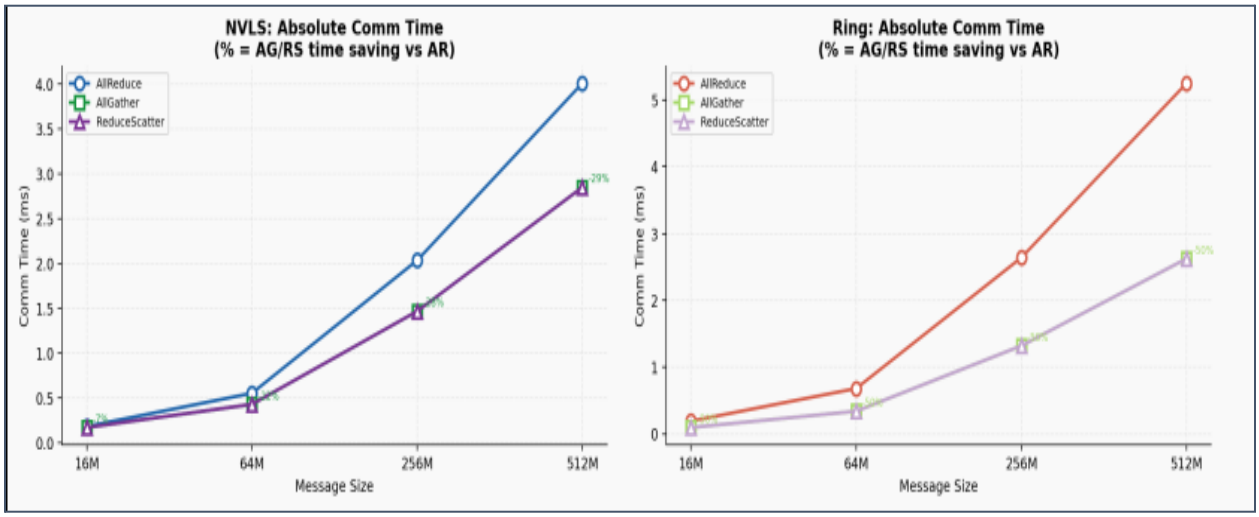


Hypothesis : Higher TP increases the frequency and size of AllReduce, thereby magnifying differences between algorithms.

7.







Hypothesis : NVLS's hardware tree acceleration is collective-specific — it wins AllReduce by 31% but loses AllGather and ReduceScatter to Ring, meaning the net benefit of NVLS in real LLM training (which uses all three) is significantly smaller than AllReduce.

10. Outcomes

The experimental results demonstrate that while NVLS provides significant acceleration for AllReduce operations, its performance is collective-specific and does not consistently outperform Ring across other communication patterns such as AllGather and ReduceScatter. As a result, the overall benefit of NVLS in realistic LLM training workloads—where multiple collectives are involved—is less pronounced than its AllReduce advantage alone would suggest.

Furthermore, increasing tensor parallelism (TP) amplifies both the frequency and size of AllReduce operations, thereby magnifying performance differences between algorithms. This highlights that communication strategy becomes increasingly critical at higher TP configurations.

The results also confirm that total execution time scales linearly with the number of layers when all other factors are held constant, aligning with theoretical expectations. Additionally, communication behavior varies with message size: NVLS exhibits higher latency compared to Ring at larger message sizes, but achieves better bandwidth scaling, indicating a trade-off between latency and throughput.

Finally, at smaller sequence lengths, communication time dominates overall execution, making the choice of communication algorithm a key factor in performance. As sequence length increases, computation becomes more significant, reducing the relative impact of communication overhead.

Overall, these findings emphasize that no single communication strategy is universally optimal; performance depends on workload characteristics such as parallelism configuration, message size, and sequence length.

11. References

- [1] SimAI paper: Wang et al., 'SimAI: Unifying Architecture Design and Performance Tuning for Large-Scale LLM Training,' NSDI 2025.
- [2] NCCL: NVIDIA Collective Communications Library. https://developer.nvidia.com/nccl
- [3] Megatron-LM: Shoeybi et al., 2020.
- [4] ASTRA-sim: https://astra-sim.github.io/tutorials/micro-2024
- [5] Thakur et al., 'Optimization of Collective Communication Operations in MPICH,' IJHPCA 2005 (original ring-allreduce analysis).

Github :
https://github.com/Vyash2002/Effect_Of_Collective_Communication_Algorithms_on_LLM_Training